Ollscoil Teicneolaíochta an Atlantaigh

Atlantic Technological University

Development of a Modular 3D Game Engine: Physics Integration

Liam Ryan, BSc (Hons) Software Development, ATU Galway.
https://github.com/ATU-Game-Engine/GameEngine.

Introduction

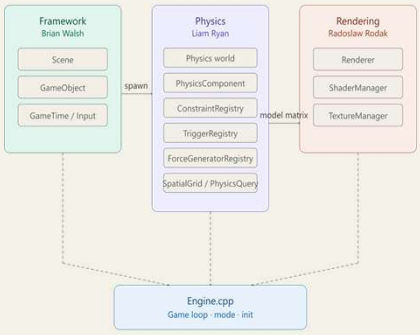
As part of a Group Final Year Project, this work focused on implementing a physics integration layer within a custom-built 3D game engine in C++. Development responsibilities were divided across3 core subsystems, the framework architecture, rendering , and physics.

The implemented physics system features rigid body dynamics, transform synchronization, a material system, spatial partitioning through a grid, raycasting, constraint handling, trigger volumes, force generation, and a scripting interface. The system is powered by Bullet Physics and exposed via a runtime editor built with Dear ImGui.

System Overview

The engine is structured as three loosely coupled subsystems coordinated by Engine.cpp. The core framework and scene manager (Brian Walsh) handles the game loop, input, object spawning, and scene serialisation. The rendering pipeline (Radoslaw Rodak) handles all visual output via OpenGL and GLSL shaders. The physics integration layer (Liam Ryan) handles rigid body simulation, collision, and all gameplay physics systems built on top of Bullet Physics.

The physics layer does not communicate directly with the renderer. Both systems share the GameObject, which holds a TransformComponent that the physics layer writes to each tick and the renderer reads from. This single shared data structure is the only coupling point between physics and rendering, keeping both subsystems independent of each other's implementation details



Tech Stack

Language: C++
Physics: Bullet Physics
Graphics API: OpenGL
Windowing/Input: GLFW
Maths: GLM
UI: Dear ImGui
Build System: CMake
Version Control: GitHub

References

Bullet Physics SDK — github.com/bulletphysics/bullet3

LearnOpenGL — learnopengl.com

Gafferon Games: Integration Basics — gafferongames.com

Physics Systems

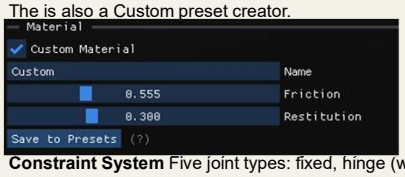
The physics integration layer is built around Bullet Physics and exposes eight systems to the rest of the engine. The fixed timestep accumulator advances simulation at 60Hz independent of render frame rate.

Three engine modes are supported, Editor (physics frozen, ImGui panels active), Game (simulation running, scripts dispatched), and Test (750 dynamic cubes spawned simultaneously to stress test simulation stability and performance). As bodies settle and sleep, Bullet's island-based sleep system reduces the active body count, and frame rate recovers automatically.

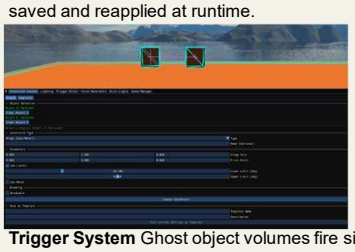
Key Features

Transform Synchronization Bidirectional sync between Bullet and the renderer. Game mode: simulation drives position. Editor mode: gizmo movements write back to the rigid body.

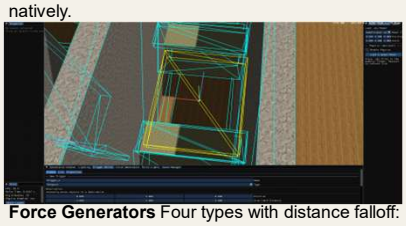
Physics Materials Eight preset surface types (Default, Wood, Metal, Rubber, Ice, Concrete, Plastic, Glass) with calibrated friction and restitution. Assignable at runtime via the ImGui material editor. The is also a Custom preset creator.



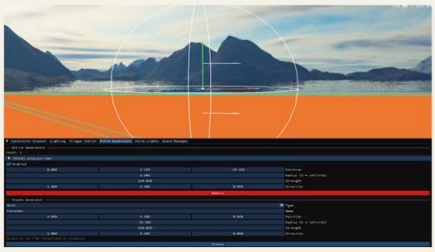
Constraint System Five joint types: fixed, hinge (with motor and angular limits), slider, spring, and 6DOF. Breakable thresholds supported. Named templates saved and reapplied at runtime.



Trigger System Ghost object volumes fire single-fire entry and exit callbacks with tag-based filtering. Frame-to-frame overlap tracking resolves the continuous callback problem absent from Bullet natively.



Force Generators Four types with distance falloff: Wind (uniform), Gravity Well (inverse-square), Vortex (orbital), and Explosion (single-frame outward impulse, auto-removed after firing).



Raycasting Wraps Bullet's rayTest API in a typed RaycastHit interface. Used for player ground detection where a downward ray provides a cleaner grounded check than Bullet contact points directly.

Spatial Grid O(1) average radius queries using spatial hashing. Objects are registered across all cells their bounding box overlaps and queries filter by actual distance after collecting candidates from covered cells.

Scripting Foundation Tag-based script attachment with four lifecycle callbacks: onStart, onUpdate, onFixedUpdate, and onDestroy. The PlayerController demonstrates the system with raycasting ground detection and fixed-timestep movement. The EnemyAI script demonstrates the system in tandem with spatial grid radius queries to detect and respond to nearby dynamic objects.

Results

Conclusion

All 13 objectives were met bidirectional transform synchronization, a physics materials system, spatial grid, raycasting, five constraint types, trigger zones with tag filtering, four force generator types, and a scripting foundation demonstrated in the GameScene,

Resulting in the successful delivery of a physics integration layer built from scratch in C++ around Bullet Physics.

Some of the bigger challenges faced throughout development included btGhostPairCallback initialisation order, which caused silent trigger failures when omitted, the continuous callback problem in the trigger system, resolved through frame-to-frame overlap tracking, and the constraint rebuild pipeline required when rigid bodies are resized at runtime. The rebuilding required careful ordering of operations to prevent crashes. Detaching constraints from the Bullet dynamics world before destroying the old rigid body, reconstructing the Bullet constraint objects against the new body using cached frame transforms, and re-registering them with the world. Incorrect ordering produced use-after-free crashes in the constraint solver, and missing frame caching caused degenerate constraint Jacobians that destabilized the simulation on the first physics step after reconstruction.

Future work includes Lua scripting integration, constraint overlap validation, and spatial grid pre-filtering for force generator pipelines

Acknowledgements

Supervised by Dr. Aaron Hurley, Atlantic Technological University Galway. Co-developers: Brian Walsh (framework) and Radoslaw Rodak (rendering).